

AvaRide

INF2007 Mobile Application Development — Project Report

Team T10 | Singapore Institute of Technology

Tushar Gupta (2400634) | Lim Tze Kai (2401009) | Ryson Toh Wei Feng (2401333) | Tan Yale (2401316)
| Nabil Bin Noor Azlan (2402978)

1. Introduction

AvaRide is a next-generation autonomous vehicle (AV) hailing application designed to eliminate the friction of traditional ride-booking. Unlike current apps that require manual input for every trip, AvaRide utilises a personal AI agent to predict user intent, automates vehicle access via secure NFC handshakes, and utilises Augmented Reality (AR) to solve the 'last-mile' problem of locating a driverless car.

The inspiration comes from the disconnect between modern autonomous hardware and legacy software interfaces. While AVs are cutting-edge, the booking experience remains stagnant — users still manually type destinations and struggle to identify their specific vehicle in crowded pickup points. AvaRide bridges this gap while relying entirely on free, open-source, and open-access services, ensuring long-term sustainability and zero licensing cost.

2. Project Idea and Motivation

Idea: AvaRide is a next-generation AV hailing application designed to bridge the interaction gap in a driverless future. It replaces legacy human-to-human coordination with a proactive Digital Concierge interface powered exclusively by free APIs and open-source libraries.

Motivation: Current market leaders (Grab, Uber) rely heavily on manual user input and human driver communication. In a driverless future, the app must bridge this communication gap without incurring API licensing costs.

Gap in Solution: AvaRide replaces paid services (Google Maps Platform, Gemini API) with free alternatives — OpenStreetMap/OSMDroid for mapping and the Groq API (LLaMA 3) for AI inference — achieving the same capability at zero cost.

Impact: This app improves societal efficiency by reducing idle time for AVs (faster boarding) and enhances individual convenience by acting as a proactive assistant rather than a passive tool.

3. Review of Similar Work

Similar Work	Key Features	Difference / Improvement in AvaRide
Grab / Uber / Gojek / TADA	Manual destination entry; 2D map for vehicle tracking.	AvaRide eliminates manual entry using the free Groq API (LLaMA 3) to predict destinations based on context (battery, time, history), reducing booking time to zero clicks.

Waymo One	Fully autonomous hailing; unlocking via app button; manually locate vehicle.	Waymo relies on plate numbers for identification. AvaRide proposes a Universal NFC Handshake that standardises secure AV access. AvaRide also introduces AR Wayfinding (ARCore — free) to visually guide users to the exact vehicle location.
Bluetooth-based car-sharing	Vehicle unlocks when phone is in Bluetooth range; relies on signal strength.	AvaRide replaces passive Bluetooth with an explicit NFC Tap-to-Ride handshake. AR wayfinding (ARCore) eliminates pickup ambiguity in crowded zones.

4. Software Architecture Design

AvaRide follows a client–cloud–vehicle architecture using exclusively free and open-source services:

- Mobile Client: UI, state management, sensors, NFC, ARCore, AI interaction
- Cloud Backend: Firebase Authentication & Firestore (free Spark plan) — real-time session synchronisation and push notifications
- External Services: OpenStreetMap / OSMDroid (routing & map display — free, open-source), Groq API with LLaMA 3 (AI inference — free tier), TomTom Traffic API (live traffic data — free tier, 2,500 req/day)
- AV Fleet: Vehicle telemetry, NFC unlock endpoint, location broadcasting

Design Justifications

- Event-driven synchronisation via Firestore enables real-time responsiveness at no cost
- Firebase Authentication (free Spark plan) provides cloud-managed secure access control
- Modular separation ensures scalability — swapping AI providers is trivial given the abstraction layer
- OpenStreetMap data is community-maintained and permanently free with no API key required for tile rendering
- Groq API provides one of the fastest free LLM inference endpoints available (LLaMA 3 70B)
- TomTom Traffic API (free tier) provides real-time traffic flow and incident data, enabling US-06 push notifications at zero cost during MVP development

5. Implementation of Key Aspects

5.1 Mobile Sensor

AvaRide uses the device's native sensor array to bridge the physical and digital states of the user. The application utilises GNSS sensors for high-frequency, real-time geospatial tracking. This continuous data stream feeds the Pick-Up Point algorithm. No paid location API is required — Android's FusedLocationProviderClient (free) provides high-accuracy location data.

5.2 Mobile Communication — NFC Tap-to-Ride

AvaRide implements a Tap-to-Ride authentication protocol using NFC (Near Field Communication). When the AV arrives, the user taps their device against the vehicle's NFC reader, initiating a cryptographically secure handshake via HMAC-SHA256 — a standard library available free in Android. The challenge-response protocol uses Host-based Card Emulation (HCE), a built-in Android feature requiring no third-party library.

For users without NFC-enabled devices, the app generates a time-boxed QR code using the free ZXing (Zebra Crossing) library as a fallback.

5.3 Mobile Multimedia — AR Wayfinding (ARCore)

To resolve navigation ambiguity in complex urban environments, AvaRide employs Augmented Reality via Google ARCore — a free SDK with no usage-based cost.

- **Precision Pick-Up:** AR overlays 3D directional markers onto the camera feed, bridging the gap between the digital map and the physical street view.
- **Vehicle Identification:** The AR interface anchors a distinct visual indicator to the specific AvaRide car, reducing boarding confusion in crowded zones.

5.4 Mobile Computing — Firebase (Free Spark Plan)

AvaRide uses Firebase Authentication and Firestore — both available on Google's permanently free Spark plan. The app employs event listeners to synchronise critical data (vehicle coordinates, trip status, ETA) in real time. Profile photos are stored in Firebase Storage (free tier: 5 GB). No paid cloud tier is required for the MVP.

5.5 Mobile AI — Groq API with LLaMA 3

AvaRide replaces the paid Gemini API with the Groq API, which offers free access to LLaMA 3 (Meta's open-source large language model). Groq's infrastructure provides industry-leading inference speeds, making it well-suited for real-time destination prediction.

- **Predictive Analytics:** Ava analyses historical travel patterns, time of day, and device telemetry to predict the user's likely destination upon app launch (e.g., suggesting 'Home' during evening commute hours), reducing the booking process to a single tap.
- **Intelligent Service Adaptation:** The agent monitors device status. Traffic conditions are sourced from the TomTom Traffic Flow API (free tier, 2,500 requests/day), which provides real-time speed and incident data. When the API detects a significant delay on the predicted route, the agent triggers a Firebase Cloud Messaging push notification suggesting an earlier departure time. Upon detecting a Low Battery state, Ava proactively notifies the user that their assigned AV is equipped with in-cabin wireless charging.

Free API / Service Summary

Component	Free Service Used	Notes
Mapping & Navigation	OpenStreetMap + OSMDroid	Open-source map tiles; no API key; Android library on Maven
AI / LLM Inference	Groq API (LLaMA 3 70B)	Free tier; fast inference; open-source model
Authentication & DB	Firebase Spark Plan	Free: 50k reads/day, 20k writes/day, 5 GB storage
AR Wayfinding	Google ARCore	Free SDK; no per-use cost

NFC / HCE	Android HCE (built-in)	No third-party library required
QR Code Fallback	ZXing (Zebra Crossing)	Open-source; free Maven dependency
Push Notifications	Firebase Cloud Messaging	Free on Spark plan
Live Traffic Data	TomTom Traffic API (free tier)	2,500 req/day free; real-time flow & incident data; no billing setup required for MVP
CI/CD Pipeline	GitHub Actions	Free for public repos; 2,000 min/month free on private

6. User Stories

ID	User Story	Acceptance Criteria
US-01	As a user, I want the app to predict my destination and pickup time based on my routine so that I can book a ride with minimal manual input.	1. Home screen displays a Suggested Trip based on current time and historical patterns. 2. Users can confirm the ride with a single tap. 3. Prediction is powered by the Groq API (LLaMA 3) analysing past trip data.
US-02	As a user, I want to use AR navigation to find the specific car assigned to me in a crowded area so that I do not waste time looking for the licence plate.	1. Users can launch an AR camera view from the Active Trip screen. 2. A 3D marker or overlay appears on top of the correct vehicle in the live camera feed. 3. The UI clearly identifies the car model and status.
US-03	As a user, I want to use an NFC handshake or QR scan to unlock the vehicle so that I can ensure a secure and seamless boarding process.	1. Vehicle doors unlock when the phone is held near the AV's NFC reader. 2. A QR code fallback (ZXing) is available if NFC is unsupported. 3. App state changes to In-Transit immediately upon successful handshake.

US-04	As a user, I want my profile, preferences, and ride history to be synced to the cloud so that I have a consistent experience across different devices.	1. User profile data (including profile photo) is stored via Firebase Storage (free tier). 2. Preferred vehicle settings are applied automatically upon booking. 3. Data persists after the user logs out and logs back in.
US-05	As a user, I want to track the real-time location of the AV and my own position on a map so that I know exactly when to head to the pickup point.	1. App uses GPS and inertial sensors (FusedLocationProviderClient) for high-accuracy tracking. 2. The OpenStreetMap tile map updates the AV's position every 5 seconds. 3. ETA updates dynamically based on the vehicle's progress.
US-06	As a user, I want the personal AI agent to notify me about external factors like bad traffic or low phone battery so that I can manage my commute safely.	1. App sends a push notification (via TomTom Traffic API + Firebase Cloud Messaging) if traffic adds >10 mins to the predicted routine. 2. A Safety Alert is triggered if battery drops below 15% during a ride. 3. AI (Groq/LLaMA 3) suggests an earlier departure time if heavy traffic is detected.

7. Product Backlog

ID	Module	Feature / User Story	Acceptance Criteria Summary	Priority	Pts
PB-01	Mobile AI	US-01: Groq API Integration	Connect to Groq API; prompt engineering for destination prediction; handle null responses.	High	8
PB-02	Sensors	US-05: OSMDroid Map Setup	Add OSMDroid dependency; configure OpenStreetMap tiles; user permission handling (Fine Location).	High	5
PB-03	Cloud	US-04: Firebase Auth & DB (Spark)	Login UI; Firestore schema for Users and Trips; Read/Write security rules.	High	3
PB-04	UI/UX	Design System Implementation	Create standardised colours, typography, and button components (Material Design 3).	High	3

PB-05	Comms	US-03: NFC / HCE Logic	Android NFC Adapter; NDEF message parsing; HMAC-SHA256 token generation via Android HCE.	Medium	5
PB-06	Comms	QR Code Fallback (ZXing)	ZXing library integration for QR generation and scanning (open-source, free).	Medium	3
PB-07	Mobile AI	US-06: Context Monitoring	Background service to monitor Battery Level and TomTom Traffic API (free tier); trigger Firebase Cloud Messaging push notifications.	Medium	5
PB-08	Multimedia	US-02: ARCore Setup	ARCore dependency; plane detection; coordinate translation (GPS to AR space).	Low	13
PB-09	Testing	Unit Testing Suite	JUnit tests for Fare Calculation and Prediction Logic (target 70% coverage).	Low	5
PB-10	DevOps	CI/CD Pipeline	GitHub Actions workflow (free) for build and linting; Firebase App Distribution setup.	High	3
PB-11	Security	Data Encryption	Encrypt user PII locally (EncryptedSharedPreferences) and in transit (TLS).	High	5
PB-12	UI/UX	UI Responsiveness	App must display a Loading/Skeleton state immediately upon request, preventing UI freeze during AI fetch.	Medium	3

8. Sprint Backlog

Sprint	Goal	Key User Stories (Backlog)
Sprint 1 (Jan 25 – Feb 20)	Core Foundation & Navigation	- US-04 (Cloud): Implement Firebase Authentication & Firestore (Spark plan) to sync user profiles. - US-05 (Sensors): Integrate OSMDroid with OpenStreetMap tiles to track user and AV positions using GPS.
Sprint 2 (Feb 21 – Mar 06)	AI Personalisation & Secure Entry	- US-01 (Mobile AI): Integrate Groq API (LLaMA 3) to analyse historical trip data and suggest zero-click bookings. - US-06 (Mobile AI): Background services for battery/TomTom traffic monitoring; push notifications via Firebase Cloud Messaging.

		• US-03 (Mobile Comms): NFC HCE handshake for secure vehicle unlocking; ZXing QR fallback.
Sprint 3 (Mar 07 – Mar 20)	AR Wayfinding & Hardening	• US-02 (Mobile Multimedia): Implement ARCore to create 3D visual overlays for locating the vehicle in crowded areas. • Security: EncryptedSharedPreferences for local PII. • Testing: JUnit suite targeting 70% coverage.

Sprint 1 Task Breakdown

Task ID	Task Description	Linked PB	Est. Hours
T1.1	Configure GitHub Actions workflow (Build & Lint) — free tier	PB-10 (DevOps)	4
T1.2	Implement Material Design 3 System (Colours / Typography)	PB-04 (UI/UX)	5
T1.3	Implement Firebase Auth Login / Register UI	PB-03 (Cloud)	8
T1.4	Define Firestore Schema & Security Rules (Spark plan)	PB-03 (Cloud)	3
T1.5	Integrate OSMDroid library & OpenStreetMap tile configuration (US-05)	PB-02 (Sensors)	6

Sprint 2 Task Breakdown

Task ID	Task Description	Linked PB	Est. Hours
T2.1	Connect Groq API (LLaMA 3) & Prompt Engineering (US-01)	PB-01 (AI)	8
T2.2	Implement Skeleton UI Loading States	PB-12 (UI/UX)	3
T2.3	Background Service for Battery / TomTom Traffic API Monitoring (US-06); Firebase Cloud Messaging push notifications	PB-07 (AI)	6

T2.4	Implement NFC Adapter, HCE Service & HMAC-SHA256 Challenge-Response (US-03)	PB-05 (Comms)	8
T2.5	Integrate ZXing (open-source) for QR Code Scanning Fallback	PB-06 (Comms)	5

Sprint 3 Task Breakdown

Task ID	Task Description	Linked PB	Est. Hours
T3.1	Add ARCore Dependencies & Plane Detection (US-02)	PB-08 (Multimedia)	8
T3.2	GPS → AR Space Coordinate Translation Logic	PB-08 (Multimedia)	8
T3.3	Implement EncryptedSharedPreferences for Local PII Encryption	PB-11 (Security)	5
T3.4	Write JUnit Tests (Target 70% Coverage)	PB-09 (Testing)	5
T3.5	Final Integration Testing & Polish	All	10

9. NFC Secure Handshake Protocol

AvaRide replaces passive proximity unlocking (e.g., Bluetooth PKES) with a deliberate NFC Tap-to-Ride handshake implemented entirely with built-in Android APIs and standard cryptographic libraries — no paid SDKs required.

End-to-End Flow

- Session creation: When the user reaches the Unlock step, the app creates a short-lived session token (5-minute TTL) tied to tripId + vehicleId, stored in Firestore.
- Physical tap: User taps phone to the AV's NFC reader (Android NFC Adapter — built-in).
- Challenge-response (HCE): Vehicle sends SELECT AID → phone confirms active session → vehicle requests challenge (GET_CHALLENGE) → phone returns nonce + timestamp → vehicle sends its nonce and requests verification (VERIFY) → phone returns HMAC(token, challenge || vehicleNonce || tripId).
- Verification: Vehicle verifies the HMAC-SHA256 proof. If valid, it unlocks the doors and signals success to the app via Firestore.
- Fallback (QR): If NFC is unavailable, the app generates a time-boxed QR code using ZXing (open-source) containing the same session token for the vehicle camera to scan.

Security Properties

- Physical proximity required (~4 cm NFC range) — relay attacks are mitigated
- Replay protection via nonce + timestamp
- Short-lived tokens (5 min) reduce attack window
- Token transmitted only as HMAC proof — never in plaintext
- No paid security SDK required — standard `javax.crypto.Mac` (HMAC-SHA256) is part of Android runtime

10. References

1. Infocomm Media Development Authority. Singapore Digital Society Report 2023. Technical report, IMDA Singapore, 2023.
2. Jiang, K., Xie, X., Dai, R., & Lu, J. Authenticated Phone Key Vulnerability in Tesla Model 3. In NDSS Symposium, 2022.
3. Li, A. Waymo One adds windshield display to distinguish fleet of identical cars. 9to5Google, 2021.
4. Liu, P. Autonomous Ridesharing: A User Experience Study on Finding and Unlocking AVs, 2025.
5. Taoglas. NFC vs Bluetooth: Understanding the Key Differences and Security for Access Control, 2024.
6. OSMDroid Contributors. OSMDroid — OpenStreetMap Tools for Android. GitHub, 2024. <https://github.com/osmdroid/osmdroid>
7. Groq. Groq API Documentation — LLaMA 3 Inference. Groq Cloud, 2024. <https://console.groq.com/docs>
8. ZXing (Zebra Crossing). Open-Source QR Code Library for Android. Apache License 2.0. <https://github.com/zxing/zxing>
9. TomTom. TomTom Traffic API Documentation — Traffic Flow and Incidents. TomTom Developer Portal, 2024. <https://developer.tomtom.com/traffic-api/documentation/traffic-flow/flow-segment-data>